

VUE.JS REACTIVITY



REACTIVE PROGRAMMING

La programmazione reattiva è un paradigma di programmazione dichiarativo. Due idee principali: **flussi di dati** (data streams) e **propagazione del cambiamento**.



REACTIVE VS IMPERATIVE

Nei linguaggi reattivi, le variabili reattive vengono ricalcolate quando una dipendenza cambia.

```
reactive var $b = a + 1
```

La variabile reattiva b cambia quando a cambia.



ESEMPIO

Linguaggio	Codice	Output Iniziale	Output Finale	Spiegazione
Imperativo	<pre>var a = 2; var b = a + 1; fmt.Println(b); a = 1; fmt.Println(b);</pre>	3	3	b non cambia, il calcolo è eseguito una sola volta.
Reattivo	<pre>var a = 2; reactive var b = a + 1; fmt.Println(b); a = 1; fmt.Println(b);</pre>	3	2	b è reattivo e si aggiorna al cambio di a.



STATO REATTIVO

Ciascun componente ha uno stato reattivo.

```
<script>
export default {
  data() {
    // Questo è lo stato reattivo
    return {
      count: 0,
    }
  }
}
</script>
<template><!-- ... --></template>
<style>/* ... */</style>
```



STATO REATTIVO

- Vue.js avvolge il valore di ritorno di `data()` nel suo sistema reattivo
- Devi dichiarare tutte le variabili reattive nell'oggetto restituito da `data()`
- Usa `undefined` o `null` se non c'è un valore
- Non usare i prefissi riservati `$` o `_`
- Vue.js è *deep-reactive*: le modifiche negli oggetti annidati vengono rilevate



ACCESSO ALLO STATO REATTIVO DA JS

Puoi accedere alla variabile reattiva da JS usando this:

```
export default {  
  data() {  
    return { count: 1 }  
  },  
  // `mounted` è un hook del ciclo di vita (li vedremo più avanti)  
  mounted() {  
    console.log(this.count) // => 1  
    // puoi cambiarla come una normale variabile  
    // questo attiverà l'aggiornamento nel DOM e  
    // tutte le computed properties che ne dipendono  
    this.count = 2  
  }  
}
```



ACCESSO ALLO STATO REATTIVO DAL TEMPLATE

Non è necessario usare la parola chiave `this` nei template:

```
<span>Conteggio: {{ count }}</span>  
<div v-text="count"></div>
```



LE VARIABILI NON SONO PURE

Nota che le variabili dalla funzione `data()` non sono "pure", sono avvolte nel sistema reattivo:

```
export default {  
  data() {  
    return {  
      someObject: {}  
    }  
  },  
  mounted() {  
    const newObject = {}  
    this.someObject = newObject  
    console.log(newObject === this.someObject) // false  
  }  
}
```



COMPUTED PROPERTIES

Le computed properties sono proprietà che si aggiornano automaticamente quando una dipendenza (variabile reattiva) cambia.

```
export default {
  data() {
    return {
      firstName: 'John',
      lastName: 'Doe'
    }
  },
  computed: {
    // Viene letto come una normale proprietà
    fullName() {
      return this.firstName + ' ' + this.lastName
    }
  }
}
```



AGGIORNAMENTI DEL DOM

Quando lo stato reattivo muta, Vue.js aggiorna il DOM automaticamente, ma **no** **sincronamente**.

I cambiamenti sono bufferizzati fino al prossimo "tick" (ciclo di update). Per accedere al DOM dopo il ciclo di update:

```
import { nextTick } from 'vue'
export default {
  // ...
  methods: {
    increment() {
      this.count++
      nextTick(() => {
        // access updated DOM
      })
    }
  }
}
```





COMPUTED CACHING

- Una proprietà **computed** è un **getter** con caching.
- Viene ricalcolata solo quando le dipendenze reattive cambiano.
- Se le dipendenze non sono cambiate, viene restituito il valore precedente **senza eseguire la funzione**.

Attenzione: Una funzione regolare definita in `methods` viene rieseguita a **ogni** re-render del componente, anche se i suoi input non sono cambiati.



SETTER

```
export default {  
  // ...  
  computed: {  
    fullName: {  
      // Getter chiamato quando viene letto this.fullName  
      get() {  
        return this.firstName + ' ' + this.lastName  
      },  
      // Setter chiamato quando viene assegnato this.fullName = '...'  
      set(newValue) {  
        // Notare che non si usa let! Si usano le proprietà reattive  
        this.firstName e this.lastName  
        [this.firstName, this.lastName] = newValue.split(' ')  
      }  
    }  
  }  
}
```

LISTENER DELLO STATO REATTIVO (WATCHERS)

- I metodi `computed` sono ottimi per calcolare valori
- I `watchers` (osservatori) sono utili per eseguire *effetti collaterali* in reazione a cambiamenti di stato.
- Sono definiti nell'oggetto `watch`



WATCHERS

- I **Watchers** vengono usati per eseguire operazioni in risposta a cambiamenti reattivi dello stato.
- Sono utilizzati per eseguire **effetti collaterali** (side effects), spesso asincroni, come fetch di dati.

```
export default {  
  data() {  
    return { count: 0 }  
  },  
  watch: {  
    // La chiave corrisponde al nome della proprietà data  
    count(newCount, oldCount) {  
      console.log(`Count cambiato da `${oldCount}` a `${newCount}``)  
      // Esegui effetti collaterali, es. salvare i dati  
    }  
  },  
  methods: {  
    increment() { this.count++ }  
  }  
}
```

```
}  
}
```

COMPUTED VS WATCHERS

Caratteristica	Computed Properties	Watchers
Scopo Primario	Calcolare un nuovo valore reattivo.	Eseguire una funzione (effetto collaterale) in risposta a un cambiamento.
Output	Restituisce un valore.	Esegue logica, non restituisce un valore.
Caching	Sì.	No.
Quando Usarlo	Per valori derivati.	Per operazioni asincrone o costose in risposta a un cambiamento.



CALL METHODS

- I metodi sono funzioni usate per eseguire logica o cambiare lo stato.
- Sono definiti nell'opzione `methods` del componente.
- Vengono chiamati da js o nel template, es. usando la direttiva `v-on` (shorthand `@`).



Richiamabili da JS

```
export default {  
  data() {  
    return { count: 0 }  
  },  
  
  methods: {  
    increment() { this.count++ }  
  },  
  
  // `mounted` is a lifecycle hook (we will see them later)  
  mounted() {  
    this.increment()  
  }  
}
```



O dal template

```
<script>
export default {
  data() {
    return { count: 0 }
  },
  methods: {
    increment() { this.count++ }
  }
}
</script>
<template>
  <button @click="increment">{{ count }}</button>
</template>
```



NON USARE ARROW NOTATION

```
Do not define arrow functions:  
export default {  
  methods: {  
    increment: () => {  
      // BAD: no `this` access here!  
    }  
  }  
}
```



DIFFERENZA TRA METODI E PROPRIETÀ CALCOLATE

- **Proprietà Calcolate (computed)**
 - Sono usate per comporre valori
 - Sono *cacheate* (memorizzate)
 - Sono ri-valutate solo quando le loro dipendenze cambiano (*lazy*)
- **Metodi (methods)**
 - Sono usati per eseguire un'azione
 - Sono ri-eseguiti ad ogni chiamata (non sono *cached*)



COMPONENTS LIFECYCLE DEFINITION

- Ogni istanza di componente attraversa diversi stati durante la sua vita.
- L'insieme di questi stati è chiamato **lifecycle** (ciclo di vita).
- Esempio di stati: `mounted`, `beforeUpdate`, `beforeUnmount`, etc.



COMPONENTS LIFECYCLE: ESECUZIONE DI CODICE

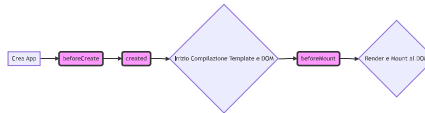
È possibile eseguire codice JS quando uno stato del ciclo di vita viene raggiunto, utilizzando gli *hook* (ganci) appropriati:

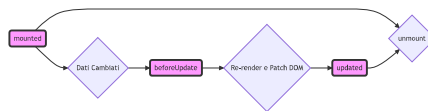
```
export default {  
  // L'hook 'mounted' è eseguito quando il componente è stato inserito nel DOM  
  mounted() {  
    console.log(`the component is now mounted.`)  
  }  
}
```

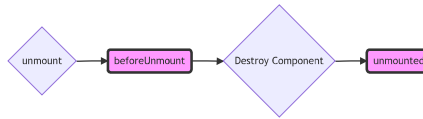


COMPONENTS LIFECYCLE DIAGRAM

Di seguito il diagramma che illustra il ciclo di vita di un componente Vue:







Esempio Pratico: Caricamento Iniziale dei Dati



Il hook più comune è `mounted()`. Viene chiamato:

1. Dopo che il componente è stato renderizzato nel DOM (Document Object

Model).

2. È il posto ideale per operazioni che richiedono l'accesso al DOM o per il **fetching dei dati** iniziali da un'API.

```
export default {
  data() {
    return {
      loading: true,
      postTitle: ''
    }
  },
  mounted() {
    // Logica eseguita DOPO che il template è stato inserito nel DOM.
    console.log('Il componente è stato montato!');

    // Esempio: Carica i dati di un post dall'API
    this.loading = true;
    fetch('https://jsonplaceholder.typicode.com/posts/1')
      .then(res => res.json())
      .then(data => {
        this.postTitle = data.title;
      })
  }
}
```

```
        this.loading = false;
    })
}
}
```

ALTRI HOOK

- `beforeCreate()`: Prima che il componente venga creato (i dati reattivi non sono ancora pronti).
- `created()`: Dopo la creazione (i dati sono pronti, ma il DOM non è ancora renderizzato). Ottimo per l'inizializzazione dei dati.
- `beforeUnmount()`: Prima che il componente venga distrutto e rimosso dal DOM. Ottimo per pulire listener o timer.
- `unmounted()`: Dopo che il componente è stato rimosso.



QUANDO USARE COSA?

Hook	Momento	Uso Comune
created	Dati pronti, DOM NO.	Inizializzazione dati.
mounted	Dati e DOM pronti.	Fetching dati (con API), interazioni DOM.
beforeUnmount	Prima della distruzione.	Pulizia di listener esterni.



REFERENCES

- <https://www.sop.inria.fr/mimosa/rp/generalPresentation/index.html>
 - <https://xgrommx.github.io/rx-book/>
 - https://en.wikipedia.org/wiki/Reactive_programming
- <https://vuejs.org/guide/essentials/reactivity-fundamentals.html>
 - <https://vuejs.org/guide/extras/reactivity-in-depth.html>





APPROFONDIMENTI



ES6 SHORTHAND METHODS

La ragione per cui non hai bisogno della parola chiave `function` è dovuta a una funzionalità introdotta in **ECMAScript 2015 (ES6)** chiamata **Sintassi Concisa dei Metodi** (*Shorthand Method Syntax*).

Questa è una caratteristica standard di JavaScript (non specifica di Vue.js), e Vue la sfrutta perché il blocco `methods` è semplicemente un *literal* (una definizione diretta) di un oggetto JavaScript standard.



SINTASSI ESTESA (PRE-ES6)

Prima di ES6, per definire una funzione come proprietà all'interno di un oggetto, si doveva usare la sintassi verbosa **chiave: valore**, dove il valore era una funzione anonima:

```
const mioOggetto = {  
  // chiave: function() { ... }  
  inc: function() {  
    this.contatore++;  
  },  
  decr: function() {  
    this.contatore--;  
  }  
};
```



SINTASSI CONCISA (ES6 E SUCCESSIVE)

ES6 ha introdotto una sintassi più breve che ti permette di definire metodi (funzioni) all'interno di un oggetto senza digitare esplicitamente `function`:

```
const mioOggetto = {  
  // chiave() { ... }  
  inc() {  
    this.contatore++;  
  },  
  decr() {  
    this.contatore--;  
  }  
};
```

Entrambe le sintassi creano esattamente lo stesso tipo di proprietà funzione sull'oggetto. La versione concisa è semplicemente più pulita e meno ripetitiva, ec

è lo standard preferito nello sviluppo JavaScript moderno.

